

ADA: Part 1 Solutions!

Student: Kyal Sin Lin Lett

ID: st126112

Part 1:

1. Correctness, Efficiency, and Ease of implementation.

2.

	Access	Search	Insert	Del	Space
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Hash Table	Best: $O(1)$ Worst: $O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$

3. The objective is to find the max subset of pairwise non-overlapping intervals from a given set of intervals.

egs $f_i \leq s_j$ or $f_j \leq s_i$

4. Earliest finish time — sort by increasing finish time, greedily select next compatible interval.

Earliest start time — sort by increasing start time, greedily select the next compatible interval.

Shortest interval — sort by increasing duration $(f_i - s_i)$ and greedily select the next compatible interval.

5. Algorithm → well-defined procedure that produces a correct answer always.

Heuristic → practical approach to find a good but not necessarily optimal solution.

Part 2:

1. Pseudocode

EFT(intervals):

Sort by finish time

selected $\leftarrow []$

last-finish $\leftarrow -\infty$

for $(s, f) \leftarrow$ sorted interval:

if $s \geq \text{last-finish}$:

selected.append $((s, f))$

last-finish $\leftarrow f$

return selected.

Algorithm:

① Sort all intervals by their finish time in ascending

② Initialise set S and initialise last-finish as $-\infty$

③ For each interval, if $s_i \geq \text{last-finish}$

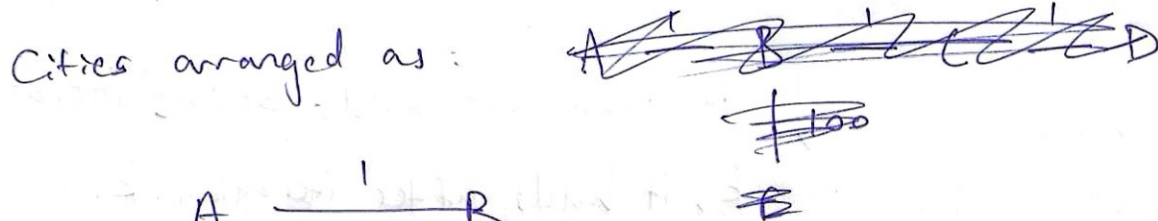
③a add (s_i, f_i) to S

③b set last-finish to f_i

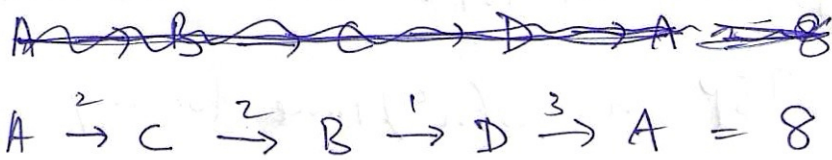
④ return S

2. NN is a heuristic because it does not guarantee to find the optimal solution. It only makes locally optimal choices without considering the global structure of the tour.

Proof by counterexample (CE)



Beginning from A, the NN heuristic of greedy approach suggests $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A = 9$. but a CE



3. Loop variant is a property that is true before the loop begins and remain true after each iteration of the loop. When loop terminates, it gives us a useful property to prove correctness.

Induction

Invariant

Base Case



Initialisation (Invariant holds before first iteration)

Inductive Hypothesis



Assume invariant holds before iteration k .

Inductive Step



if invariant holds before iteration k , it holds after iteration k .

Conclusion



when loop terminates, invariant condition proves correctness.

4. NP-hard means that the TSP problem is at least as hard as the hardest problems in NP. There is no known algorithm that solves TSP in $O(n^k)$ time for any constant k .

5. Modularity improves implementation by decomposition of tasks into separate reusable functions. This reduces code duplication and improves testability.

It also helps with maintenance because you can modify.

~~rewriting~~ a module without ~~replacing~~ ^{rewriting} the entire ~~algorithm~~ codebase.

Part 3:

1. Proof: Assume for contradiction that EFT is suboptimal, such that it selects fewer intervals than optimal. Eg:

EFT selects: $g_1, g_2, \dots, g_k$ where $(f(g_1) \leq f(g_2) \leq \dots \leq f(g_k))$

OPT selects: $o_1, o_2, \dots, o_m$ where $(f(o_1) \leq f(o_2) \leq \dots \leq f(o_m))$

where $m > k$.

Lemma: $\forall i$ where $1 \leq i \leq k$: $f(g_i) \leq f(o_i)$

Proof of lemma by induction:

BC: ($i=1$) g_1 has the minimum finish time among intervals

so $f(g_1) \leq f(o_1)$

Inductive step: Assume $f(g_j) \leq f(o_j) \forall j < i$

Since o_i is compatible with o_{i-1} , we have

$$s(o_i) \geq f(o_{i-1})$$

By Ind. Hypo. $f(g_{i-1}) \leq f(o_{i-1})$, thus ~~$s(o_i) \geq f(o_{i-1})$~~

$s(o_i) \geq f(g_{i-1})$ meaning o_i is compatible with g_{i-1}

Thus, EFT chooses earliest compatible finishing interval: $f(g_i) \leq f(o_i)$

Since $m > k$, interval o_{k+1} exists

By lemma, $f(g_k) \leq f(o_k)$. Since o_{k+1} is compatible with o_k

in OPT: $s(o_{k+1}) \geq f(o_k)$. Combining them, $s(o_{k+1}) \geq f(o_k) \geq f(g_k)$

$\therefore O_{k+1}$ is compatible with g_k

This means EFT should have also selected O_{k+1} as it's $k+1^{\text{th}}$ interval in the return set. But EFT stopped at k intervals only, this is a contradiction

$\therefore$ the assumption of $m > k$ is false, since OPT is optimal,

$$m \geq k \text{ or } k = m$$

2. ~~Claim~~ Claim: In greedy approach, SD provides the optimal solution:

Suppose SD provides the optimal solution, then SD yields the max subset of non-overlapping intervals.

given example intervals: $\{(0,3), (3,5), (5,8)\}$

SD chooses $\{(3,5)\}$ as the duration 2 is minimum.

OPT chooses $\{(0,3), (5,8)\}$ as they don't overlap

This shows that SD did not choose the most optimal solution, thereby the claim is false.

3. For $n = 20$

$$n! = 20! \approx 2.43 \times 10^{18}$$

$$\text{For a tour (circular)} = \frac{(n-1)!}{2} = \frac{19!}{2} \approx 6.08 \times 10^{16}$$

This becomes impractical because assume:

computer evals 10^9 permutations/s

$$\therefore \text{time taken (s)} = \frac{2.43 \times 10^{18}}{10^9} = 2.43 \times 10^9 \text{ seconds} \\ \approx 77 \text{ years}$$

$$\begin{aligned} \text{time taken for a tour} &= \frac{6.08 \times 10^{16}}{10^9} = 6.08 \times 10^7 \text{ seconds} \\ &\approx 2 \text{ years.} \end{aligned}$$

Asymptotic analysis:

$n!$ grows faster than exponential, say 2^n

let $a_n = \frac{2^n}{n!}$, using ratio test:

$$\begin{aligned} \frac{a_{n+1}}{a_n} &= \frac{2^{n+1}/(n+1)!}{2^n/n!} = \frac{2^{n+1}}{(n+1)!} \cdot \frac{n!}{2^n} \\ &= \frac{2^{n+1}}{2^n} \cdot \frac{n!}{(n+1)!} \\ &= 2 \cdot \frac{n!}{(n+1) \cdot n!} = \frac{2}{n+1} \end{aligned}$$

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \lim_{n \rightarrow \infty} \frac{2}{n+1} = 0$$

4. Trial-and-error is insufficient because we cannot test all possible ~~inputs~~ inputs. Passing tests may not guarantee correctness since a counterexample edge case might be the $n+1^{\text{th}}$ case.

Math. induction is preferred because it is a universal proof and proves correctness for all valid inputs. It is rigorous and builds on established mathematics.

5.

	Quicksort	Mergesort
Best-case	$O(n \log n)$	$O(n \log n)$
Avg. case	$O(n \log n)$	$O(n \log n)$
Worst case	$O(n^2)$	$O(n \log n)$
Space Complexity	$O(\log n)$	$O(n)$ due to auxiliary array

Quicksort: General purpose use where memory constraints must be considered

Mergesort: Cases where stability in performance is required and external sorting requirements like file sorting.